

Real-Time Classroom Monitoring

A Python system for real-time data collection, stream processing and anomaly detection

STUDENT	Yerlan Utepbergenov
COURSE	Real-Time Data Analysis with Python
COVERAGE	Weeks 1–4
TOOLS	Python · Pandas · NumPy · Matplotlib
DATE	October 2026

180

readings processed

4

real-time indicators

4

alert rules

98

alerts raised

Abstract

This project implements a small but complete **real-time data processing system** in Python. A simulated classroom sensor produces a new reading every minute (temperature, humidity and CO₂). The system receives each reading **as soon as it arrives**, cleans it, updates live statistics over a rolling window, checks it against four alert rules, shows the results in a continuously refreshed table, and stores everything for later analysis. During a simulated three-hour session, 180 readings were processed and 98 alerts were raised. CO₂ stayed above the recommended 1000 ppm limit for 81 minutes during the class, and all five injected sensor glitches were caught by the spike rule.

1 Introduction

Many modern systems need to react to data *while it is being produced*, not hours later: factories monitor machines, banks watch transactions, and smart buildings control heating and ventilation. This kind of work is called **real-time data analysis**. Instead of loading a complete dataset and analysing it once, a real-time system handles a never-ending stream of records, one at a time, and must keep its results up to date after every new record.

The goal of this midterm project is to build such a system using the tools covered in Weeks 1–4 of the course: Python generators for streaming, incremental processing, and Pandas and NumPy for cleaning, statistics and aggregation. The objectives were to:

- define a realistic streaming scenario with a clear data source, schema and frequency;
- build a reproducible **data generator** that behaves like a live sensor;
- process records **incrementally**, as they arrive;
- calculate at least three **real-time indicators**;
- implement **rule-based anomaly detection**;
- display **continuously updated results** and **store** them for reproducibility.

2 Scenario and Data

The chosen domain is a **smart campus**. A sensor installed in a university classroom measures the indoor climate and sends a new reading **every 60 seconds**. Facilities staff want to know immediately when the air quality gets bad (high CO₂ makes students tired and less focused), when the room is too hot, and when the sensor itself is misbehaving.

No physical sensor was available, so the readings are produced by a Python **generator** that simulates one. The simulation covers 09:00–12:00 (180 readings) and includes a class from 09:30 to 11:00, during which body heat and breathing push the temperature and CO₂ up. A fixed random seed makes every run produce exactly the same data, so all results in this report can be reproduced.

Data schema

Field	Type	Unit	Description
timestamp	datetime	–	Time the reading was taken (one per minute)
temperature	float	°C	Room air temperature
humidity	float	%	Relative humidity
co2	float	ppm	CO ₂ concentration; outdoor air is about 420 ppm

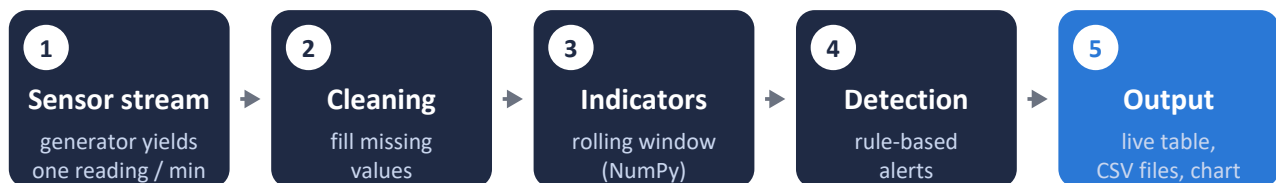
Table 1: Structure of one sensor reading

Real sensors are never perfect, so the generator deliberately adds two kinds of data-quality problems that the system must handle:

- **Missing values** (about 3% of readings): the temperature arrives as `None`, as if the message was partly lost.
- **Spikes** (about 3% of readings): the temperature suddenly jumps by +8 °C for one minute, imitating an electrical glitch in the sensor.

3 System Architecture

The system is a simple **pipeline** of five stages. Every reading passes through all stages before the next reading is requested, which is what makes the processing incremental.



each reading passes through all five stages before the next one arrives

Figure 1: Processing pipeline

Stage	Python component	Responsibility
1. Data source	<code>sensor_stream()</code>	Generator that <i>yields</i> one reading at a time, like a live feed
2. Cleaning	<code>clean()</code>	Replaces a missing temperature with the last valid value and marks it
3. Indicators	<code>indicators()</code>	Moving average, standard deviation, min and max over the last 10 readings
4. Detection	<code>detect()</code>	Checks four rules and returns a list of alerts
5. Output	main loop	Refreshes a live table every 10 readings; saves CSV files and a chart

Table 2: Components of the system

4 Implementation

4.1 Stream generator

The data source is written as a Python **generator function**. Because it uses `yield` instead of `return`, it does not build the whole dataset in memory. It produces one reading, pauses, and continues only when the consumer asks for the next one. This is exactly how a real data feed behaves.

```
def sensor_stream(n=N_READINGS, seed=42):
    rng = np.random.default_rng(seed)
    start = pd.Timestamp("2026-10-05 09:00")
    temp, co2 = 21.0, 450.0
    for i in range(n):
        in_class = 30 <= i < 120 # students are in the room
        temp += 0.1 * ((24.5 if in_class else 21.0) - temp) + rng.normal(0, 0.1)
        co2 += 0.1 * ((1200 if in_class else 450) - co2) + rng.normal(0, 10)
        reading = {"timestamp": start + pd.Timedelta(minutes=i),
                  "temperature": round(temp, 2), "humidity": ..., "co2": round(co2, 1)}
        if rng.random() < 0.03: reading["temperature"] += 8 # spike
        if rng.random() < 0.03: reading["temperature"] = None # missing value
        yield reading
```

Temperature and CO₂ move gradually towards a target value: a higher one while students are present and a lower one when the room is empty. Small random noise makes the signal look natural.

4.2 Incremental acquisition and processing

The main loop consumes the generator one reading at a time. A `deque(maxlen=10)` keeps only the ten most recent temperatures. When a new value is added, the oldest one is dropped automatically, so memory use stays constant no matter how long the stream runs. A short `time.sleep()` simulates the waiting time between real messages.

```
window = deque(maxlen=WINDOW)
results = []
for reading in sensor_stream():
    reading = clean(reading, last_temp) # 1. clean
    alerts = detect(reading, list(window)) # 2. check rules
    window.append(reading["temperature"])
    reading.update(indicators(window)) # 3. update statistics
    reading["alerts"] = ", ".join(alerts)
    results.append(reading) # 4. store
    last_temp = reading["temperature"]
    if len(results) % 10 == 0: # 5. refresh live table
        clear_output(wait=True); display(pd.DataFrame(results).tail(5))
    time.sleep(DELAY)
```

Cleaning. If the temperature is missing, `clean()` fills it with the last valid value (forward fill) and sets a missing flag. The record is kept instead of thrown away, so the timeline stays complete and the problem stays visible.

4.3 Real-time indicators

After every reading, NumPy calculates the following statistics over the rolling window of the last ten minutes:

Indicator	Calculation	Why it is useful
Moving average	<code>np.mean(window)</code>	Smooths out noise and shows the real trend
Rolling standard deviation	<code>np.std(window)</code>	Shows how stable the signal is; jumps when something unusual happens
Rolling minimum / maximum	<code>np.min / np.max</code>	Range of recent values
Alert count	running total	How many problems have been detected so far

Table 3: Real-time indicators

4.4 Rule-based anomaly detection

The detector checks every reading against four simple, easy-to-explain rules. A reading can trigger several alerts at once.

Alert	Rule	Meaning
HIGH_TEMP	temperature > 27 °C	Room is too hot
HIGH_CO2	CO ₂ > 1000 ppm	Poor ventilation, so a window should be opened
SPIKE	temperature – moving average > 3 °C	Sudden jump, probably a sensor error
MISSING_VALUE	temperature was None	Data was lost and had to be filled in

Table 4: Alert rules

The spike rule compares the new reading with the average of the *previous* readings, before it is added to the window. This way a bad value cannot hide itself by pulling the average towards it. A fixed 3 °C threshold was chosen over a z-score because it worked more reliably. When several spikes happen close together, the standard deviation grows and a z-score test starts missing them.

4.5 Live display and storage

Every ten readings the notebook clears the previous output and draws a fresh table with the latest five records and the total number of alerts. This gives a dashboard that updates in place. When the stream ends, Pandas saves the results to three files:

- `results.csv`: every reading with all indicators and alerts (180 rows);
- `alerts.csv`: only the readings that triggered at least one alert (86 rows);
- `results.png`: the chart of temperature and CO₂.

5 Results

The full stream of 180 readings was processed in about ten seconds of wall-clock time. Table 5 summarises the processed data.

Statistic	Temperature (°C)	Humidity (%)	CO ₂ (ppm)	Moving avg (°C)	Rolling std (°C)
Mean	22.84	39.84	827.4	22.84	0.71
Std. deviation	2.10	0.98	338.8	1.67	1.05
Minimum	20.65	37.30	419.5	20.83	0.00
Median	22.66	39.80	825.5	22.84	0.14
Maximum	32.40	42.90	1232.4	25.93	3.45

Table 5: Summary statistics of the processed stream

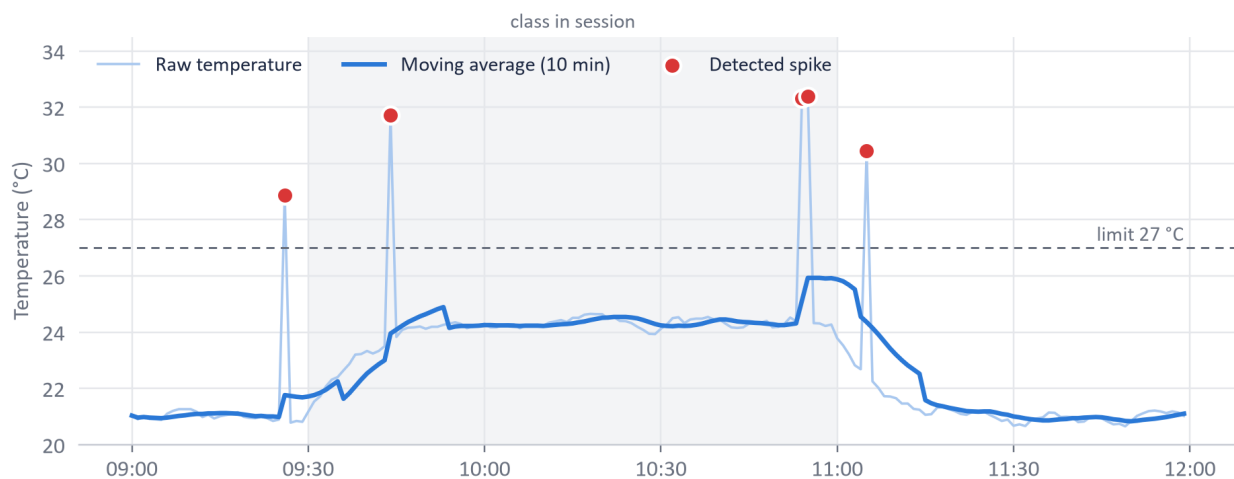


Figure 2: Temperature readings, the 10-minute moving average, and detected spikes

Figure 2 shows that the temperature rises from about 21 °C to 24.5 °C during class and returns to normal afterwards. The moving average follows this trend smoothly while ignoring the noise. All five spikes (red dots) stand out clearly and were detected, including two that happened in consecutive minutes at 10:54 and 10:55.

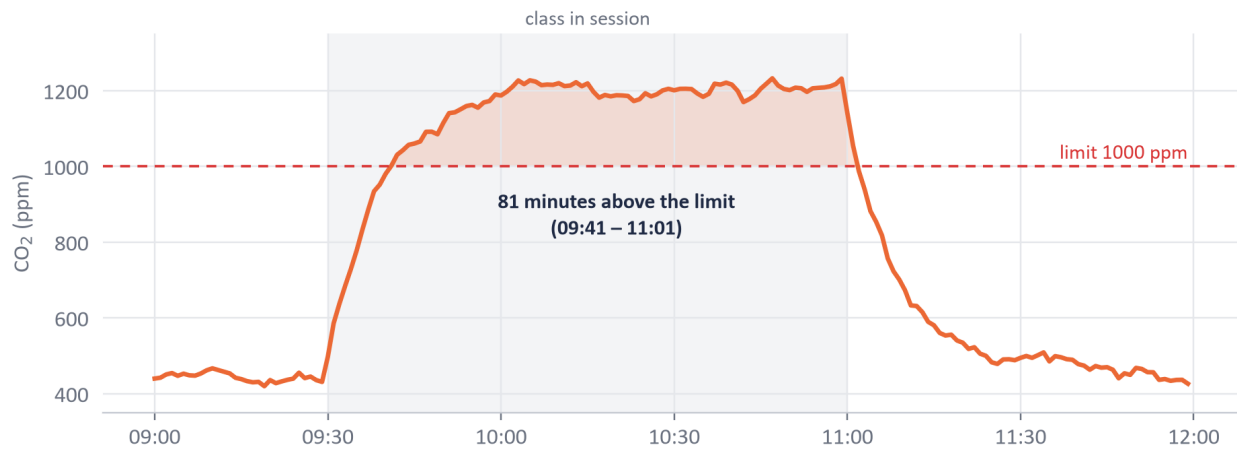


Figure 3: CO₂ concentration compared with the 1000 ppm ventilation limit

CO₂ reacts strongly to the class (Figure 3). About eleven minutes after the students arrive it crosses 1000 ppm, and it stays above the limit until the room empties. In total the room had poor air quality for **81 minutes**, almost the entire lesson.

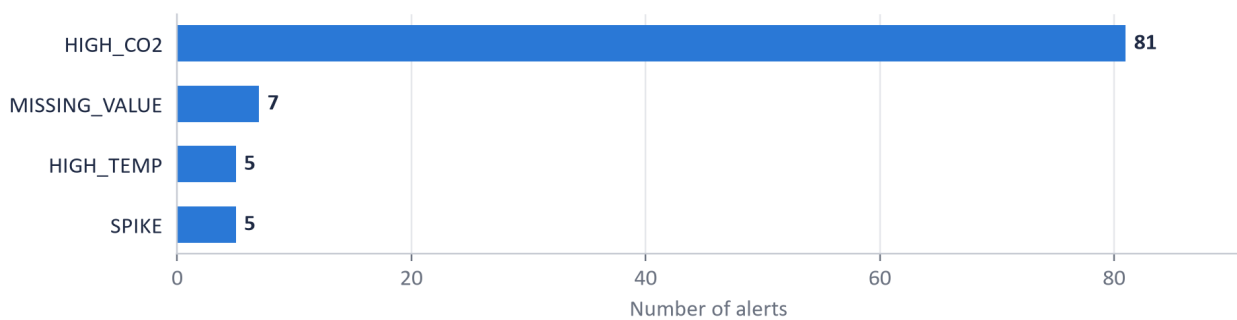


Figure 4: Number of alerts by type

Time	Temperature	Moving avg	Alerts raised
09:26	28.86 °C	21.76 °C	HIGH_TEMP, SPIKE
09:44	31.73 °C	23.95 °C	HIGH_TEMP, HIGH_CO2, SPIKE
10:54	32.32 °C	25.12 °C	HIGH_TEMP, HIGH_CO2, SPIKE
10:55	32.40 °C	25.93 °C	HIGH_TEMP, HIGH_CO2, SPIKE
11:05	30.45 °C	24.36 °C	HIGH_TEMP, SPIKE

Table 6: All temperature anomalies detected during the stream

6 Discussion

Key finding: the room needs better ventilation

CO₂ crossed 1000 ppm only 11 minutes after the lesson started and stayed above the limit for 81 minutes, almost the whole class. Opening a window or turning on ventilation as soon as the system raises its first HIGH_CO2 alert would keep the air fresher and help students stay focused.

Combining rules gives better answers. Every one of the five HIGH_TEMP alerts happened at the same moment as a SPIKE alert (Table 6), and the moving average never reached 27 °C (its maximum was 25.9 °C). This tells us the room never actually overheated. These were sensor glitches, not a real problem. A system with only a temperature threshold would have sent five false alarms. Looking at the two rules together lets us separate *real events* from *bad data*.

Observation: spikes affect the moving average

A single +8 °C spike raises the 10-minute average by almost 1 °C for ten minutes (visible as small steps in Figure 2). A moving median, or leaving detected spikes out of the window, would make this indicator more robust.

Limitations and possible improvements

- **Alert flooding:** HIGH_CO2 is raised every minute while the level stays high (81 alerts). Raising one alert when the problem starts and another when it ends would be more practical.
- **Simulated data:** the generator could be replaced by a real sensor, an MQTT broker or a public API without changing the rest of the pipeline, because the main loop only needs an iterable.
- **Fixed thresholds:** limits such as 27 °C could adapt to the season or to each room.
- **One sensor:** the same design scales to many rooms by keeping one window per sensor.

7 Conclusion

The project shows a working real-time data pipeline built only with Python, Pandas and NumPy. Readings are generated and processed one at a time, live statistics are updated after every record, problems are detected with clear rules, and the results are displayed live and saved for later analysis. Even in this small example, real-time analysis produced a useful insight (the classroom needs more ventilation) and showed how combining simple rules can tell real events apart from sensor errors.

Requirements checklist

Requirement	How it is met	
Realistic scenario with source, schema and frequency	Smart-classroom sensor, 1 reading/min (Section 2)	✓
Timestamped data and a stream generator	sensor_stream() generator with fixed seed	✓
Incremental data acquisition	Records processed one at a time in the main loop	✓
Pandas and NumPy for analysis	NumPy rolling statistics; Pandas tables, CSV, summaries	✓
At least 3 real-time indicators	Moving average, std, min/max, alert count	✓
At least 1 rule-based detector	4 rules: HIGH_TEMP, HIGH_CO2, SPIKE, MISSING_VALUE	✓
Continuously updated display	Live table refreshed every 10 readings	✓
Stored results	results.csv, alerts.csv, results.png	✓
At least one table and one visualization	Tables 1–6, Figures 1–4	✓

How to run

Install the requirements with `pip install -r requirements.txt`, open `midterm.ipynb` in VS Code or Jupyter, select the virtual-environment kernel and choose **Run All**. The stream takes about ten seconds and then produces all tables, the chart and the output files.